
Attaques et défense des API

OWASP API Security Top 10, scénarios d'attaque et sécurisation multi-framework

Gana Faye

Sécurité des applications web | 24 juillet 2026

API SECURITY

Objectifs du module

À la fin, vous saurez :

- ▶ cartographier une API et comprendre son architecture ;
- ▶ expliquer les dix risques OWASP API Security 2023 ;
- ▶ reconnaître les scénarios d'attaque techniques et métier ;
- ▶ tester autorisation, entrées, ressources et dépendances ;
- ▶ sécuriser une API avec plusieurs frameworks modernes ;
- ▶ intégrer contrôles, tests et surveillance au cycle DevSecOps.

Fil conducteur

DÉCOUVRIR



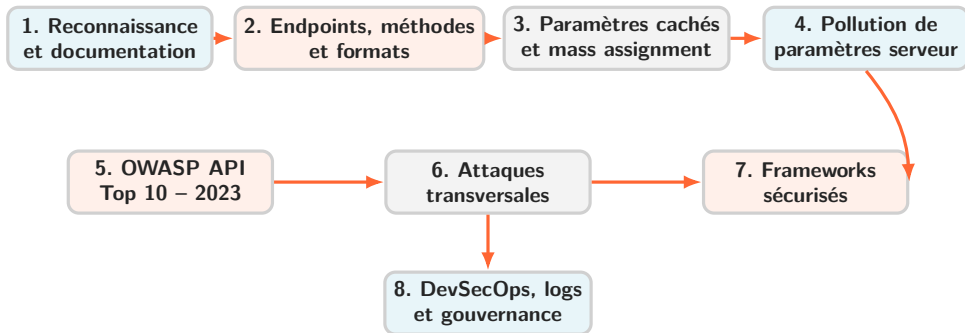
COMPRENDRE



TESTER ET CORRIGER

Cadre éthique

Les manipulations offensives sont réservées aux laboratoires et aux systèmes explicitement autorisés.



PARTIE 1

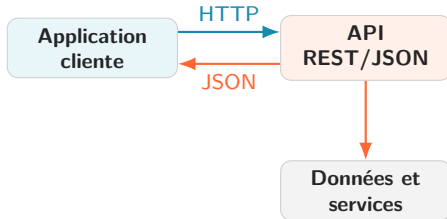
Reconnaissance des API

Qu'est-ce qu'une API ?

Définition

Une API est un contrat permettant à deux composants logiciels d'échanger des données et de déclencher des opérations.

- ▶ **Client** : navigateur, mobile, service ou script.
- ▶ **Endpoint** : adresse d'une fonction exposée.
- ▶ **Requête** : méthode, chemin, paramètres, en-têtes, corps.
- ▶ **Réponse** : statut, en-têtes et données.



Point sécurité

L'interface graphique ne montre qu'une partie des capacités. L'API constitue souvent la véritable surface d'attaque.

Concept	Exemple	Question de sécurité
Ressource	/api/users/42	Puis-je changer l'identifiant ?
Collection	/api/orders	La liste est-elle filtrée par utilisateur ?
Méthode	GET, POST, PATCH	D'autres méthodes sont-elles acceptées ?
Paramètre	?status=pending	Existe-t-il des paramètres non documentés ?
Représentation	JSON, XML, formulaire	Le parseur change-t-il les contrôles ?
Version	/api/v1/	Une ancienne version reste-t-elle exposée ?

Réflexe

Ne pas tester uniquement les valeurs : tester aussi les chemins, méthodes, formats, versions et relations entre objets.

Qu'est-ce que l'API Recon ?

La reconnaissance API consiste à construire une carte fiable de l'interface :

- ▶ hôtes et préfixes d'API ;
- ▶ versions et environnements ;
- ▶ endpoints publics et authentifiés ;
- ▶ méthodes HTTP réellement acceptées ;
- ▶ paramètres visibles ou cachés ;
- ▶ schémas de données et relations d'autorisation.

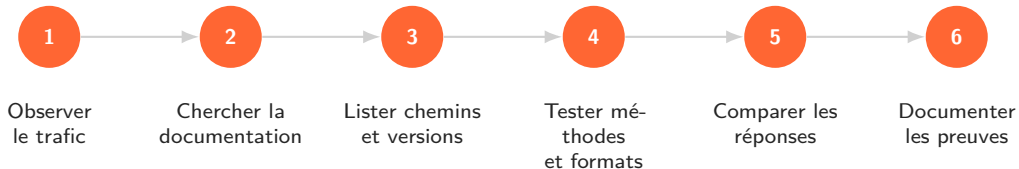
Livrable : inventaire testable

Endpoint	/api/v1/users/{id}
Méthodes	GET, PATCH
Auth.	cookie de session
Entrées	id, email, role
Formats	JSON
Risque	contrôle d'accès, mass assignment

Erreur fréquente

Scanner sans comprendre produit beaucoup de bruit et peut manquer la logique métier.

Méthode de reconnaissance en six étapes



Principe expérimental

Modifier un seul élément à la fois et conserver une requête témoin. La différence de statut, taille, structure ou délai devient alors interprétable.

Sources passives

- ▶ historique Proxy et outils développeur ;
- ▶ JavaScript et fichiers source map ;
- ▶ applications mobiles et clients officiels ;
- ▶ erreurs, liens et en-têtes de réponse ;
- ▶ documentation publique et dépôts autorisés.

Sources actives

- ▶ variations contrôlées de chemins ;
- ▶ tests de méthodes HTTP ;
- ▶ découverte par listes de mots ;
- ▶ comparaison des codes et longueurs ;
- ▶ essais de paramètres et types de contenu.

Précaution

La découverte active peut créer une charge importante ou déclencher des actions. Commencer par des méthodes sûres et respecter le périmètre.

PARTIE 2

Documentation des API

Pourquoi la documentation est une mine d'informations

Une documentation peut révéler :

- ▶ les chemins et versions disponibles ;
- ▶ les méthodes et codes de réponse ;
- ▶ les paramètres obligatoires ou facultatifs ;
- ▶ les schémas JSON complets ;
- ▶ les mécanismes d'authentification ;
- ▶ des fonctions anciennes absentes de l'interface.

Deux formats

LISIBLE PAR L'HUMAIN

Swagger UI, Redoc, portail développeur.

LISIBLE PAR MACHINE

OpenAPI JSON/YAML, WSDL, collections.

Attention

« Documenté » ne signifie pas « autorisé pour tous ». Chaque opération doit encore appliquer ses contrôles d'accès.

Découvrir la documentation

Emplacements courants à tester dans un laboratoire autorisé :

```
/api  
/api/  
/docs  
/api/docs  
/swagger  
/swagger-ui  
/swagger/index.html  
/openapi.json  
/openapi.yaml  
/v1/api-docs  
/v2/api-docs  
/v3/api-docs
```

Autres indices

Un lien dans le HTML, une variable JavaScript, un en-tête d'erreur ou une réponse 302 peut révéler l'emplacement réel.

API Users	
GET	/api/users/{id}
POST	/api/users
PATCH	/api/users/{id}
DEL	/api/users/{id}

Une interface interactive permet souvent :

- ▶ d'ouvrir un endpoint ;
- ▶ de voir ses paramètres ;
- ▶ d'afficher un exemple ;
- ▶ parfois d'envoyer une requête.

Risque

Une fonction sensible peut être directement testable depuis le navigateur si la documentation est exposée.

```
openapi: 3.0.3
paths:
  /api/users/{id}:
    patch:
      parameters:
        - name: id
          in: path
      requestBody:
        content:
          application/json:
            schema:
              $ref: "#/components/schemas/User"
```

Ce que le fichier révèle

- ▶ endpoint PATCH ;
- ▶ paramètre de chemin id ;
- ▶ format JSON ;
- ▶ schéma User réutilisable.

Utilisation

Importer la définition dans un outil compatible permet de générer une collection de requêtes de base.

- 1 Intercepter une action légitime et identifier le préfixe /api.
- 2 Rechercher une documentation sous le même hôte.
- 3 Repérer un endpoint sensible absent de l'interface, par exemple une suppression.
- 4 Construire la requête conformément au schéma documenté.
- 5 Envoyer d'abord une requête non destructive ou visant une ressource de laboratoire.
- 6 Vérifier le statut, le corps et l'effet métier.



Leçon

La vulnérabilité n'est pas l'existence de la documentation, mais l'absence de contrôle d'accès sur l'opération révélée.

Automatiser

- ▶ extraire les couples chemin/méthode ;
- ▶ générer des requêtes valides ;
- ▶ repérer les champs obligatoires ;
- ▶ comparer documentation et trafic réel.

Ne pas automatiser aveuglément

- ▶ les actions destructives ;
- ▶ la création massive d'objets ;
- ▶ les tests hors périmètre ;
- ▶ les valeurs métier irréversibles.

Bonne pratique

Ajouter à chaque requête générée le bon contexte d'authentification, puis vérifier manuellement les contrôles d'objet et de fonction.

PARTIE 3

Endpoints, méthodes et formats

Anatomie d'un endpoint

https

schéma

api.exemple.test

hôte

/v1/users

version + ressource

/42

identifiant

?expand=role

paramètre

Axes de variation

Version, casse, pluriel, identifiant, extension, barre finale, sous-ressource et paramètres peuvent révéler des comportements différents.

Observer l'application

- ▶ connexion et profil ;
- ▶ recherche et filtres ;
- ▶ panier, paiement, fichiers ;
- ▶ administration ;
- ▶ appels périodiques en arrière-plan.

Inspecter les ressources

- ▶ chaînes `/api/` dans le JavaScript ;
- ▶ fonctions `fetch` et clients HTTP ;
- ▶ routes dans les source maps ;
- ▶ noms de modèles et paramètres ;
- ▶ anciennes versions.

Indice fort

Un endpoint appelé uniquement après une condition d'interface peut rester accessible même si le bouton correspondant est caché.

Interagir avec un endpoint

```
GET /api/v1/users/42?expand=role HTTP/2
Host: api.exemple.test
Authorization: Bearer <jeton>
Accept: application/json
```

Ce que l'on modifie

chemin, identifiant, paramètres, méthode, en-têtes, corps et format.

Ce que l'on mesure

statut, taille, structure JSON, message, délai, en-têtes et effet métier.

Règle

Rejouer d'abord la requête sans modification : elle devient la référence de comparaison.

Méthode	Intention habituelle	Test utile
GET	Lire une ressource	Modifier identifiant et expansion
POST	Créer ou déclencher	Ajouter des champs inattendus
PUT	Remplacer entièrement	Envoyer un objet complet modifié
PATCH	Modifier partiellement	Tester des propriétés sensibles
DELETE	Supprimer	Vérifier l'autorisation fonctionnelle
OPTIONS	Décrire des capacités	Comparer avec les méthodes réelles
HEAD	Métadonnées sans corps	Détecter une ressource avec prudence

Identifier les méthodes réellement prises en charge

- 1 Observer les méthodes utilisées par le client officiel.
- 2 Consulter la documentation et les en-têtes Allow.
- 3 Tester OPTIONS comme indice, sans le considérer comme une preuve complète.
- 4 Remplacer manuellement la méthode sur une requête valide.
- 5 Adapter le corps et le type de contenu à la nouvelle méthode.
- 6 Vérifier l'effet métier, pas seulement le code HTTP.

Pourquoi tester ?

Un routeur peut accepter PATCH ou DELETE même si l'interface n'utilise que GET. Les contrôles peuvent aussi différer selon la méthode.

Réponses fréquentes

- ▶ 200/201/204 : opération traitée ;
- ▶ 400/422 : entrée comprise mais invalide ;
- ▶ 401 : authentification requise ;
- ▶ 403 : action refusée ;
- ▶ 404 : absent ou volontairement masqué ;
- ▶ 405 : méthode non permise.

Pièges

- ▶ un 200 peut contenir une erreur ;
- ▶ un 403 peut confirmer l'existence ;
- ▶ un 404 peut masquer un défaut d'autorisation ;
- ▶ un 500 peut révéler un parseur ou un champ interne.

Conclusion

Toujours corréler statut, corps, en-têtes et état de l'application.

Identifier les types de contenu

Content-Type	Forme	Risque à tester
application/json	objet structuré	champs cachés, types, doublons
application/x-www-form-urlencoded	paires clé/valeur	paramètres dupliqués
multipart/form-data	champs + fichiers	métadonnées et noms de parties
application/xml	arbre XML	parseur et structure alternative
text/plain	texte brut	parseur tolérant ou conversion implicite

Test contrôlé

Changer Content-Type, convertir le corps et comparer la validation. Deux parseurs différents peuvent produire deux objets différents.

Même intention, deux représentations

```
POST /api/profile HTTP/2
Content-Type:
  application/json

{
  "displayName": "Alice",
  "newsletter": true
}
```

```
POST /api/profile HTTP/2
Content-Type:
  application/x-www-form-urlencoded

displayName=Alice&
newsletter=true
```

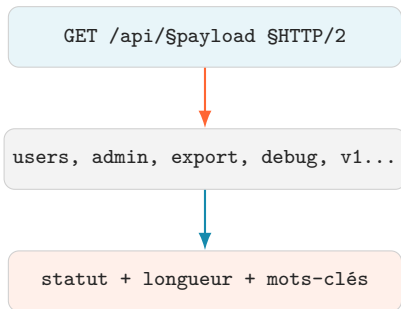
Question

Les deux chemins de désérialisation appliquent-ils exactement la même liste de champs autorisés, les mêmes conversions et les mêmes contrôles ?

- 1 Repérer un endpoint lié à une fonction visible, par exemple `/api/products/1`.
- 2 Chercher des variantes : collection, version antérieure, sous-ressource, action.
- 3 Tester une liste de noms avec Intruder à faible débit.
- 4 Classer les réponses par statut, longueur et structure.
- 5 Rejouer les anomalies dans Repeater.
- 6 Vérifier si l'endpoint inutilisé applique encore authentification et autorisation.

Exemples de variantes

`/api/products` `/api/v1/products` `/api/internal/products` `/api/products/export`



Réglages

- ▶ attaque Sniper ;
- ▶ liste contextuelle ;
- ▶ débit raisonnable ;
- ▶ suivi des redirections maîtrisé ;
- ▶ filtres sur statut et taille.

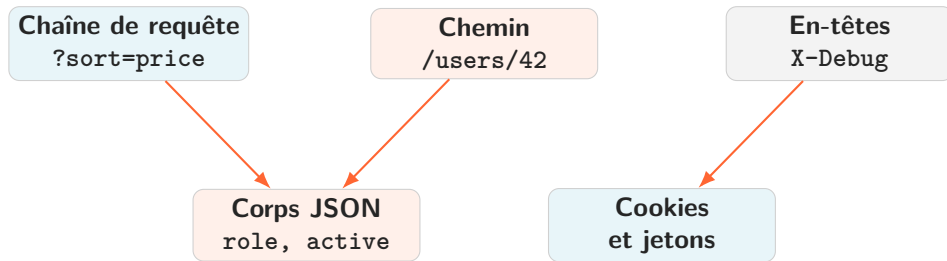
Validation

Une réponse différente est un candidat, pas une vulnérabilité. Il faut l'analyser manuellement.

PARTIE 4

Paramètres cachés et affectation de masse

Où se cachent les paramètres ?



Paramètre caché

Entrée acceptée par le serveur mais absente de l'interface ou de la documentation destinée à l'utilisateur.

Sources

- ▶ réponses JSON plus riches que les requêtes ;
- ▶ erreurs mentionnant un champ ;
- ▶ JavaScript et schémas OpenAPI ;
- ▶ anciennes versions ;
- ▶ noms de propriétés métier.

Méthodes

- ▶ ajout manuel dans Repeater ;
- ▶ Intruder avec liste de noms ;
- ▶ extension de découverte de paramètres ;
- ▶ comparaison de réponse ;
- ▶ variation du type de la valeur.

Signal

Une erreur passe de « propriété inconnue » à « valeur invalide » : le nom du champ est probablement reconnu.

Indicateur	Requête témoin	Avec paramètre candidat
Statut	200	400, 403 ou 200
Taille	742 octets	781 octets
Message	aucun	« invalid role »
Structure	objet standard	champ supplémentaire
Effet métier	inchangé	propriété modifiée

Meilleure preuve

Lire de nouveau la ressource après la modification. Un changement persistant confirme que le serveur a traité le champ.

Affectation de masse : définition

Une affectation de masse se produit lorsqu'un framework copie automatiquement les propriétés reçues vers un objet interne.

Vulnérabilité

Le client peut fournir une propriété sensible que l'interface ne devait jamais lui permettre de modifier.

- ▶ role
- ▶ isAdmin
- ▶ price
- ▶ approved
- ▶ ownerId

```
{"name":"A","role":"admin"}
```

Désérialisation automatique

Objet utilisateur
role = admin

```
PATCH /api/profile HTTP/2  
Content-Type: application/json
```

```
{  
  "displayName": "Alice"  
}
```

```
HTTP/2 200 OK
```

```
{  
  "id": 42,  
  "displayName": "Alice",  
  "email": "a@test",  
  "role": "user",  
  "isAdmin": false  
}
```

Hypothèse de test

Les propriétés `role` et `isAdmin` apparaissent dans le modèle retourné. Sont-elles aussi acceptées en entrée ?

Identifier les paramètres cachés sensibles

- 1 Créer ou modifier un objet avec l'interface normale.
- 2 Comparer les champs envoyés et les champs retournés.
- 3 Repérer les propriétés calculées, internes ou privilégiées.
- 4 Ajouter une seule propriété candidate avec une valeur non destructive.
- 5 Observer erreurs, normalisation et persistance.
- 6 Tester l'autorisation avec un compte à faibles privilèges.

Différence essentielle

Un champ accepté n'est vulnérable que si son traitement permet une action que l'utilisateur ne devrait pas effectuer.

```
PATCH /api/profile HTTP/2
Content-Type: application/json

{
  "displayName": "Alice",
  "isAdmin": true
}
```

Résultat sûr

Le serveur ignore ou rejette isAdmin, puis retourne un état inchangé.

Résultat vulnérable

La propriété est persistée et accorde un privilège ou modifie une donnée protégée.

Vérification

Relire le profil et tester une fonction réservée. Ne pas conclure à partir du seul écho JSON.

- 1 Intercepter la mise à jour légitime d'un objet.
- 2 Examiner la réponse pour découvrir des propriétés supplémentaires.
- 3 Ajouter la propriété sensible à la requête de mise à jour.
- 4 Choisir une valeur contrôlée conforme au laboratoire.
- 5 Envoyer la requête et relire la ressource.
- 6 Démontrer l'impact métier attendu par le TP.

MODÈLE RETOURNÉ



CHAMP CACHÉ



PRIVILÈGE

Au niveau du code

- ▶ utiliser des DTO d'entrée dédiés ;
- ▶ autoriser explicitement les champs modifiables ;
- ▶ séparer modèles publics et internes ;
- ▶ refuser les propriétés inconnues ;
- ▶ convertir les types strictement.

Au niveau métier

- ▶ contrôler chaque propriété sensible ;
- ▶ journaliser les changements de privilège ;
- ▶ exiger une action administrative distincte ;
- ▶ tester les comptes à faibles privilèges ;
- ▶ maintenir les schémas de validation.

Anti-modèle

Copier directement tout le JSON reçu dans une entité de base de données.

PARTIE 5

Pollution des paramètres côté serveur

Définition de la pollution côté serveur

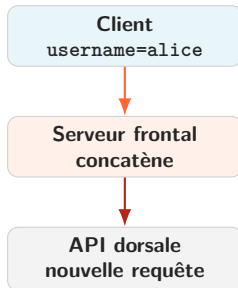
La *Server-Side Parameter Pollution* (SSPP) apparaît lorsque l'entrée utilisateur est insérée dans une seconde requête construite par le serveur.

Problème

Des séparateurs injectés modifient la structure de la requête interne : ajout, suppression ou remplacement de paramètres.

À ne pas confondre

La pollution côté serveur modifie une requête créée en interne ; la pollution HTTP classique envoie directement des paramètres dupliqués au même serveur.



Construction vulnérable d'une chaîne de requête

```
# Entree recue
username = "alice"

# Requete interne
GET /users/search?
  name=alice&
  publicProfile=true
```

```
# Entree injectee
username =
  "alice&role=admin"

# Requete interne
GET /users/search?
  name=alice&
  role=admin&
  publicProfile=true
```

Cause

Le serveur concatène la chaîne reçue au lieu d'utiliser un constructeur d'URL qui encode chaque valeur.

- 1 Identifier une entrée transmise à un service dorsal.
- 2 Établir une réponse témoin stable.
- 3 Tester un séparateur encodé, par exemple %26.
- 4 Ajouter un paramètre invalide pour provoquer une différence.
- 5 Tester un nom de paramètre plausible.
- 6 Essayer une troncature encodée si le contexte le permet.
- 7 Comparer les réponses et confirmer sans action destructive.

Encodage

Le séparateur doit souvent être encodé dans la requête externe afin d'être décodé avant la construction de la requête interne.

Tronquer une chaîne de requête

Entree externe:

alice%23

Après decodage:

alice#

Requete interne construite:

GET /users/search?name=alice#&publicProfile=true

Effet recherché

Le caractère # introduit un fragment. Selon le composant qui construit et envoie l'URL, la partie suivante peut ne plus être transmise au service dorsal.

Nuance

Le comportement varie selon les bibliothèques, les couches de décodage et le protocole interne. Il doit être vérifié expérimentalement.

Injection d'un paramètre invalide

```
username=alice%26xyz=123
```

Requete interne supposee:

```
GET /users/search?name=alice&xyz=123&publicProfile=true
```

Si rien ne change

L'entrée est peut-être encodée correctement, ignorée ou non transmise au backend.

Si une erreur apparaît

Le service dorsal peut avoir interprété xyz comme un nouveau paramètre.

But

Confirmer la possibilité de modifier la structure avant de rechercher un paramètre valide.

```
username=alice%26field=email
```

Requete interne:

```
GET /users/search?name=alice&field=email&publicProfile=true
```

- ▶ Le nom peut provenir d'une erreur, de la documentation ou d'une réponse.
- ▶ Une différence structurée est plus probante qu'un simple changement de taille.
- ▶ Tester plusieurs valeurs permet de comprendre la fonction du paramètre.

Risque

Un paramètre valide peut demander un champ secret, changer le périmètre d'une recherche ou sélectionner une action interne.

Remplacement d'un paramètre existant

Requête interne :

```
role=user&role=admin
```

Le résultat dépend de la règle du parseur :

- ▶ première valeur gagnante ;
- ▶ dernière valeur gagnante ;
- ▶ tableau de valeurs ;
- ▶ concaténation ;
- ▶ rejet du doublon.

First wins : user

Last wins : admin

Array : [user,admin]

Reject : erreur

Conséquence

Deux composants qui appliquent des règles différentes peuvent créer un contournement de validation.

- 1 Identifier une fonction qui interroge une API interne, par exemple recherche ou récupération de compte.
- 2 Injecter un séparateur encodé suivi d'un paramètre invalide.
- 3 Déduire, à partir de l'erreur, la structure de la requête interne.
- 4 Tester un paramètre valide découvert dans le message ou le comportement.
- 5 Utiliser une troncature si un paramètre imposé empêche le résultat attendu.
- 6 Extraire uniquement la preuve demandée par le laboratoire.

Chaîne de raisonnement

ERREUR CONTRÔLÉE



STRUCTURE INTERNE



PARAMÈTRE VALIDE

Le serveur peut insérer une entrée dans un chemin plutôt que dans une query string :

```
GET /api/users/{username}/profile
```

Entrées à étudier

/, ., .., %2F, %252F, point-virgule et suffixes attendus.

Risques

- ▶ changement de ressource ou de sous-ressource ;
- ▶ suppression d'un suffixe imposé ;
- ▶ normalisation différente entre proxy et backend ;
- ▶ accès à une route interne voisine.

Exemple de chemin REST construit côté serveur

Entree:

username = "alice"

Requete interne:

GET /internal/users/alice/profile

Entree manipulee:

username = "../admin"

Apres normalisation possible:

GET /internal/admin/profile

Méthode

Tester d'abord des variations bénignes et comparer les messages. La normalisation peut intervenir dans le frontal, le routeur, le proxy ou le backend.

Dans un objet JSON ou XML, la structure peut être modifiée par :

- ▶ ajout de propriétés inattendues ;
- ▶ propriétés dupliquées ;
- ▶ modification de type : chaîne, tableau, objet, booléen ;
- ▶ fermeture d'un objet puis ajout d'un autre champ ;
- ▶ imbrication destinée à un service dorsal ;
- ▶ conversion entre formats appliquant des règles différentes.

Question centrale

Quel objet exact chaque couche produit-elle après parsing, validation, conversion puis sérialisation ?

JSON : propriétés dupliquées

```
{  
  "username": "alice",  
  "role": "user",  
  "role": "admin"  
}
```

Frontal

Peut valider la première valeur : user.

Backend

Peut conserver la dernière valeur : admin.

Défense

Rejeter les clés dupliquées et utiliser des parseurs/configurations cohérents entre toutes les couches.

JSON imbriqué : confusion de schéma

```
{
  "profile": {
    "displayName": "Alice"
  },
  "options": {
    "includePrivate": true
  }
}
```

- ▶ Le frontal peut valider uniquement profile.
- ▶ Une fusion générique peut transmettre aussi options.
- ▶ Le backend peut reconnaître une option non documentée.

Lien avec la mass assignment

Les deux familles se recoupent : une structure injectée peut devenir une propriété interne automatiquement affectée.

Utiles pour

- ▶ listes de chemins ;
- ▶ noms de paramètres ;
- ▶ méthodes et types de contenu ;
- ▶ variations encodées ;
- ▶ tri des réponses.

Insuffisants pour

- ▶ comprendre l'effet métier ;
- ▶ choisir une preuve sûre ;
- ▶ distinguer écho et persistance ;
- ▶ confirmer une autorisation ;
- ▶ gérer les actions irréversibles.

Outils typiques

Repeater pour raisonner, Intruder pour varier, Comparer pour mesurer, extensions de découverte pour suggérer.



- ▶ limiter le débit et le nombre de charges utiles ;
- ▶ exclure les méthodes destructives de la phase de découverte ;
- ▶ journaliser chaque variation ;
- ▶ arrêter en cas d'effet inattendu ;
- ▶ confirmer dans Repeater avec une preuve minimale.

Construction sûre

- ▶ utiliser un constructeur d'URL ;
- ▶ encoder chaque valeur au bon moment ;
- ▶ ne jamais concaténer l'entrée brute ;
- ▶ utiliser des appels typés entre services ;
- ▶ rejeter les paramètres inconnus.

Parsing cohérent

- ▶ définir une politique de doublons ;
- ▶ rejeter les clés JSON dupliquées ;
- ▶ normaliser une seule fois ;
- ▶ partager les schémas de validation ;
- ▶ tester les conversions de format.

Défense en profondeur

Le service dorsal doit appliquer ses propres contrôles d'autorisation, même si le frontal affirme avoir validé la requête.

PARTIE 6

OWASP API Security Top 10 – 2023

Pourquoi un Top 10 spécifique aux API ?

Les API exposent directement :

- ▶ des identifiants d'objets et des données structurées ;
- ▶ des fonctions métier appelables sans interface ;
- ▶ des traitements automatisables à grande échelle ;
- ▶ des dépendances entre services internes et tiers ;
- ▶ plusieurs versions et clients simultanés.

Conséquence

Le contrôle d'accès, les limites de consommation et l'inventaire deviennent aussi importants que la validation syntaxique.

Référence

L'OWASP API Security Top 10 2023 est un document de sensibilisation et de priorisation, pas une liste exhaustive de tests.

Autorisation et identité

- API1** BOLA : objets d'autres utilisateurs
- API2** Broken Authentication
- API3** Broken Object Property Level Authorization
- API5** Broken Function Level Authorization

Abus métier et ressources

- API4** Unrestricted Resource Consumption
- API6** Sensitive Business Flows

Architecture et exploitation

- API7** Server-Side Request Forgery
- API8** Security Misconfiguration

Gouvernance et dépendances

- API9** Improper Inventory Management
- API10** Unsafe Consumption of APIs

Évolution 2023

L'exposition excessive et l'affectation de masse sont regroupées dans API3 autour de l'autorisation au niveau des propriétés.

Définition

L'API vérifie que l'utilisateur est connecté, mais pas qu'il peut accéder à l'objet désigné par l'identifiant fourni.

Scénario

GET /api/invoices/1042 devient /1043 et retourne la facture d'un autre client.

Tests

- ▶ remplacer ID numérique, UUID ou slug ;
- ▶ utiliser deux comptes propriétaires différents ;
- ▶ tester lecture, modification et suppression ;
- ▶ vérifier les sous-ressources.

Défense

Charger l'objet dans le périmètre autorisé : propriétaire, organisation, tenant et rôle.

BOLA : exemple vulnérable et correction

```
// Vulnerable
Order order = repo.findById(id)
    .orElseThrow();
return order;
```

```
// Safer
Order order = repo
    .findByIdAndOwnerId(id, user.id())
    .orElseThrow(NotFound::new);
return order;
```

Règle

L'identifiant n'est jamais une preuve d'autorisation. La requête de données doit intégrer le périmètre de l'identité courante.

- ▶ mots de passe faibles ou absence de limitation ;
- ▶ jetons prévisibles, trop longs ou non révoqués ;
- ▶ validation JWT incomplète ;
- ▶ récupération de compte fragile ;
- ▶ authentification différente selon les endpoints ;
- ▶ clés API exposées dans le client ou les journaux.

Attaques

Credential stuffing, brute force distribué, vol de jeton, replay, confusion de clé ou d'émetteur JWT.

Défense

MFA pour les actions sensibles, limitation contextuelle, rotation/révocation, validation complète des jetons et journalisation.



Erreurs dangereuses

Accepter un algorithme inattendu, ignorer l'audience, faire confiance aux claims sans politique locale ou placer un secret durable dans le payload.

Lecture excessive

L'API retourne des propriétés sensibles que le client masque ensuite : salaire, rôle, jeton interne, données personnelles.

Écriture excessive

Le client modifie une propriété sensible par affectation de masse : `role`, `approved`, `price`.

Test

Comparer les champs envoyés, retournés et documentés. Ajouter une propriété candidate puis relire l'objet.

Défense

DTO distincts de lecture/écriture, allowlist de champs, sérialisation selon le rôle et autorisation par propriété.

Ressources visées :

- ▶ CPU, mémoire, stockage et bande passante ;
- ▶ requêtes base de données coûteuses ;
- ▶ envoi d'e-mails, SMS ou appels facturés ;
- ▶ tailles de page et exports ;
- ▶ uploads, transformations d'images et IA ;
- ▶ profondeur et batching GraphQL.

Attaque

Demander `limit=100000`, lancer de nombreux exports ou répéter une opération coûteuse à faible débit.

Défense

Quotas par identité, limites de taille/temps/coût, pagination bornée, files de travaux et budgets métier.

Définition

Un utilisateur atteint une fonction réservée à un rôle supérieur : administration, export global, remboursement, suspension ou changement de rôle.

Scénarios

POST /api/admin/users accessible à un client normal ; remplacement de GET par DELETE ; ancienne route /v1/admin encore active.

Défense

Politique deny-by-default, autorisation par route et méthode, rôles/scopes côté serveur, tests de matrice d'accès.

Question	BOLA – API1	BFLA – API5
Cible	un objet précis	une fonction/route
Exemple	facture d'un autre client	endpoint de remboursement
Variation	identifiant	méthode, chemin, rôle
Contrôle attendu	propriété/tenant	permission fonctionnelle
Preuve	accès à l'objet interdit	exécution de l'action interdite

L'opération est autorisée individuellement, mais son automatisation massive détruit le modèle métier :

- ▶ réservation ou achat de stock rare ;
- ▶ création massive de comptes ;
- ▶ collecte de coupons ou bonus ;
- ▶ scraping de prix ou de disponibilités ;
- ▶ spam d'invitations ou de commentaires.

Particularité

Ce risque n'est pas forcément une erreur d'autorisation : la fonction légitime est utilisée à une échelle ou une vitesse abusive.

Défense

Limites métier, détection de bots, preuve de travail adaptée, délais, quotas par compte/appareil et surveillance comportementale.

Une API récupère une URL contrôlée : avatar distant, webhook, import, prévisualisation, PDF, stock ou proxy.

Attaque

Forcer le serveur à joindre 127.0.0.1, une adresse privée, un service cloud de métadonnées ou un système dorsal.

Défense

- ▶ allowlist de destinations ;
- ▶ validation après DNS ;
- ▶ revalidation des redirections ;
- ▶ blocage des plages réservées ;
- ▶ egress filtering ;
- ▶ authentification des services internes.

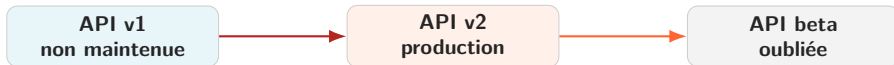
- ▶ mode debug ou stack traces ;
- ▶ CORS trop permissif ;
- ▶ TLS ou en-têtes faibles ;
- ▶ méthodes inutiles exposées ;
- ▶ stockage cloud public ;
- ▶ comptes et secrets par défaut ;
- ▶ documentation administrative publique.

Exemple CORS

Réfléchir n'importe quelle origine tout en autorisant les credentials peut exposer les données d'un utilisateur connecté.

Défense

Configuration déclarative, profils séparés, scanners de configuration, secrets gérés et durcissement automatisé.



Risques

Anciennes versions, hôtes de test, shadow APIs, documentation obsolète et propriétaires inconnus.

Défense

Catalogue automatique, propriétaire identifié, dates de fin de vie, télémétrie, suppression effective et contrôle du déploiement.

Idée

Une API fait confiance aux données d'un fournisseur, d'un partenaire ou d'un service interne comme si elles étaient sûres.

Scénarios

Injection dans une réponse partenaire, redirection non contrôlée, fichier malveillant, données excessives, webhook forgé ou dépendance compromise.

Défense

Valider les réponses tierces, limiter délais/taille/redirections, vérifier signatures de webhooks, isoler les traitements et appliquer le moindre privilège.

PARTIE 7

Attaques transversales contre les API

Les données peuvent atteindre :

- ▶ requêtes SQL ou opérateurs NoSQL ;
- ▶ commandes système ;
- ▶ moteurs de templates ;
- ▶ annuaires LDAP ;
- ▶ expressions et moteurs de recherche.

Exemple NoSQL

Un objet reçu à la place d'une chaîne peut introduire un opérateur de comparaison si le schéma n'est pas strict.

Défense

Requêtes paramétrées, schémas stricts, allowlists, aucune commande construite par concaténation et privilèges minimaux.

```
// Dangerous  
String sql = "SELECT * FROM users "  
+ "WHERE email='" + email + "'";
```

```
// Safer  
SELECT * FROM users  
WHERE email = ?  
  
bind(1, email)
```

Important

La validation de format améliore la robustesse, mais ne remplace jamais la paramétrisation de la requête.

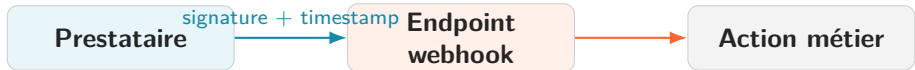
- ▶ introspection et schéma exposé ;
- ▶ requêtes profondes ou très larges ;
- ▶ aliases et batching pour multiplier les actions ;
- ▶ autorisation oubliée dans un resolver ;
- ▶ N+1 et épuisement de ressources ;
- ▶ erreurs trop détaillées.

Défense

Coût maximal, profondeur, nombre d'aliases, pagination, autorisation dans chaque resolver, requêtes persistées et limitation par identité.

Piège

Limiter uniquement le nombre de requêtes HTTP ne suffit pas si une requête contient cent opérations.



- ▶ vérifier la signature sur le corps brut, avec comparaison constante ;
- ▶ vérifier timestamp, identifiant d'événement et fenêtre de rejeu ;
- ▶ rendre le traitement idempotent ;
- ▶ ne pas faire confiance au seul filtrage IP ;
- ▶ traiter en file avec privilèges minimaux.

Attaques

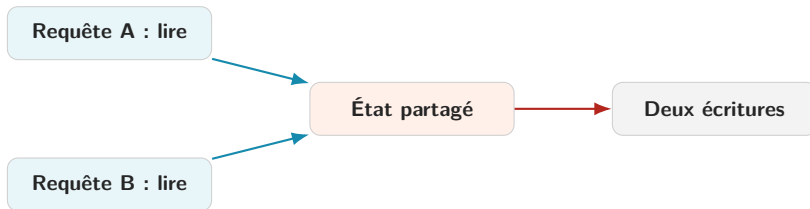
Extension trompeuse, type MIME forgé, fichier polyglotte, archive explosive, path traversal, métadonnées malveillantes et traitement vulnérable.

Défense

Taille bornée, nom généré, stockage hors webroot, détection réelle du format, ré-encodage, antivirus/sandbox, archive limitée et téléchargement avec en-têtes sûrs.

Scénario

Deux requêtes concurrentes lisent le même état « coupon disponible » puis appliquent toutes deux la réduction avant l'enregistrement.



Défense

Transactions, contraintes uniques, verrouillage adapté, opérations atomiques et clés d'idempotence.

- ▶ ne jamais mettre en cache une réponse privée sous une clé publique ;
- ▶ intégrer les dimensions nécessaires : identité, autorisation, tenant, langue ;
- ▶ définir explicitement `Cache-Control` et `Vary` ;
- ▶ éviter les réponses sensibles sur des endpoints partagés ;
- ▶ purger lors des changements d'autorisation ;
- ▶ tester les CDN, gateways et caches applicatifs ensemble.

Risque

Une API correctement autorisée à l'origine peut fuiter si un cache intermédiaire sert la réponse d'un autre utilisateur.

Risque	Quand ?	Défense
CORS	lecture cross-origin autorisée	allowlist exacte des origines
CSRF	authentification par cookie	token CSRF + SameSite + origine
XSS client	jeton accessible au JS	CSP, sortie sûre, durée courte
Token leak	URL, logs ou Referer	Authorization header, redaction

Nuance

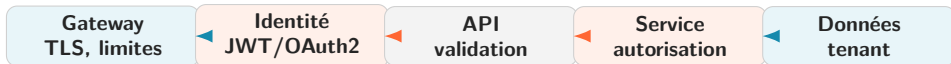
Une API utilisant uniquement un bearer token explicitement ajouté par le client n'a pas le même modèle CSRF qu'une API authentifiée automatiquement par cookie.



- ▶ utiliser plusieurs comptes, rôles, tenants et états métier ;
- ▶ varier objet, fonction, propriété, méthode et format ;
- ▶ tester limites, concurrence, dépendances et anciennes versions ;
- ▶ confirmer l'effet métier et conserver une preuve minimale.

PARTIE 8

Sécuriser les API par framework



Principe

La gateway réduit l'exposition, mais l'API et le service métier restent responsables de l'autorisation sur chaque objet, fonction et propriété.

```
@Bean
SecurityFilterChain api(HttpSecurity http) throws Exception {
    return http
        .csrf(csrf -> csrf.disable())
        .sessionManagement(s -> s.sessionCreationPolicy(STATELESS))
        .authorizeHttpRequests(a -> a
            .requestMatchers("/actuator/health").permitAll()
            .requestMatchers(HttpMethod.DELETE, "/api/users/**")
                .hasAuthority("SCOPE_users:delete")
            .anyRequest().authenticated())
        .oauth2ResourceServer(o -> o.jwt(Customizer.withDefaults()))
        .build();
}
```

À compléter

Configurer l'issuer, valider audience/scopes et ne pas utiliser la simple présence du JWT comme autorisation métier.

```
public record ProfileUpdate(  
    @Size(max=80) String displayName,  
    @Email String email  
) {}  
  
@PreAuthorize("@orders.canRead(#id, authentication)  
    ")  
@GetMapping("/api/orders/{id}")  
OrderView get(@PathVariable UUID id) { ... }
```

- ▶ DTO sans role ni ownerId;
- ▶ Bean Validation sur l'entrée;
- ▶ contrôle objet avant accès;
- ▶ projection de sortie limitée;
- ▶ gestion uniforme des erreurs.

- ▶ Spring Security Resource Server pour JWT ou jetons opaques ;
- ▶ méthodes sécurisées avec `@PreAuthorize` et politiques testées ;
- ▶ DTO séparés, validation et rejet des champs inattendus selon le mapper ;
- ▶ Actuator limité à un réseau/role dédié ;
- ▶ CORS explicite et profils de configuration séparés ;
- ▶ limites au proxy/gateway et dans les services coûteux ;
- ▶ logs sans jetons, mots de passe ni données sensibles.

```
Route::middleware(['auth:sanctum', 'throttle:api'])
->patch('/orders/{order}', [OrderController::class, 'update']);

public function update(UpdateOrderRequest $request, Order $order) {
    $this->authorize('update', $order);
    $order->update($request->validated());
    return new OrderResource($order);
}
```

Chaîne sûre

Authentifier avec Sanctum, autoriser avec une Policy, valider via Form Request et sérialiser avec une API Resource.

```
class User extends Model {  
    protected $fillable = [  
        'display_name', 'email'  
    ];  
}
```

```
$data = $request->validate([  
    'display_name' => ['string','max:80'],  
    'email' => ['email'],  
]);  
$user->update($data);
```

À éviter

`$user->update($request->all())` et un modèle dont toutes les propriétés sont affectables.

- ▶ abilities/scopes Sanctum pour les jetons API ;
- ▶ Politiques et Gates pour chaque ressource et fonction ;
- ▶ rate limiters nommés selon le coût métier ;
- ▶ validation des fichiers et stockage privé ;
- ▶ URLs signées pour les opérations temporaires ;
- ▶ tâches en queue idempotentes ;
- ▶ désactivation du debug en production et gestion sûre des secrets.

```
oauth2 = OAuth2PasswordBearer(
    tokenUrl="token",
    scopes={"orders:read": "Read orders"}
)

@app.get("/orders/{order_id}")
def read_order(
    order_id: UUID,
    user: User = Security(current_user, scopes=["orders:read"]),
    db: Session = Depends(get_db)
):
    return get_owned_order(db, order_id, user.id)
```

Principe

Le scope autorise la fonction ; la requête de données limite ensuite l'objet au propriétaire ou au tenant.

```
class UserUpdate(BaseModel):  
    model_config = ConfigDict(extra="forbid")  
    display_name: str | None = Field(None,  
        max_length=80)  
    email: EmailStr | None = None
```

```
class UserOut(BaseModel):  
    id: UUID  
    display_name: str  
    # no password_hash  
    # no internal_role
```

Bénéfice

`extra="forbid"` rejette les propriétés inattendues et les modèles de sortie évitent l'exposition accidentelle.

- ▶ valider signature, issuer, audience, expiration et scopes des JWT ;
- ▶ hacher les mots de passe avec un algorithme adapté ;
- ▶ limiter tailles de corps et uploads au proxy et dans l'application ;
- ▶ configurer précisément CORSMiddleware ;
- ▶ ne pas exposer docs OpenAPI en production si elles ne sont pas nécessaires ;
- ▶ utiliser timeouts et allowlists pour les appels sortants ;
- ▶ déployer plusieurs workers avec limites et observabilité.

```
app.patch('/api/orders/:id', authenticate, async (req, res) => {
  const input = OrderUpdateSchema.parse(req.body);
  const order = await db.order.findFirst({
    where: { id: req.params.id, ownerId: req.user.id }
  });
  if (!order) return res.sendStatus(404);
  res.json(await updateOrder(order.id, input));
});
```

À ajouter

Helmet, CORS précis, limite de corps, rate limiting distribué, gestion centralisée des erreurs et requêtes paramétrées via ORM/driver.

```
builder.Services.AddAuthentication().AddJwtBearer();
builder.Services.AddAuthorization(o =>
    o.AddPolicy("OrdersRead", p => p.RequireClaim("scope", "orders:read")));

[Authorize(Policy = "OrdersRead")]
[HttpGet("{id:guid}")]
public async Task<ActionResult<OrderDto>> Get(Guid id) {
    var order = await db.Orders
        .SingleOrDefaultAsync(x => x.Id == id && x.OwnerId == UserId);
    return order is null ? NotFound() : Ok(mapper.Map<OrderDto>(order));
}
```

PHP

PDO avec requêtes préparées, validation stricte, password_hash, secrets hors code, routeur deny-by-default, réponses JSON sans erreurs internes.

Java

PreparedStatement, validation Bean Validation, bibliothèque JWT maintenue, contrôles d'autorisation centralisés, sérialisation avec DTO et timeouts HTTP.

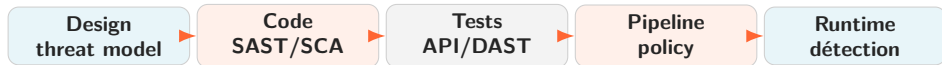
Message

Sans framework, il faut reconstruire consciemment toutes les garanties : parsing, auth, autorisation, erreurs, limites, CORS, logs et mises à jour.

Besoin	Spring Boot	Laravel	FastAPI
Auth API	Resource Server	Sanctum/Passport	OAuth2 dependencies
Autorisation	@PreAuthorize	Policies/Gates	Depends/Security
Validation	Bean Validation	Form Request	Pydantic
Sortie sûre	DTO/projection	API Resource	response model
Rate limit	gateway/filters	RateLimiter	proxy/middleware
Mass assignment	DTO + mapper	\$fillable + validated	extra forbid

PARTIE 9

DevSecOps, gouvernance et exploitation sûre



- ▶ tests contractuels OpenAPI et tests négatifs d'autorisation ;
- ▶ analyse de dépendances et images ;
- ▶ secrets détectés avant commit ;
- ▶ infrastructure/configuration contrôlée comme du code ;
- ▶ blocage du déploiement si une route n'a pas de propriétaire.

Journaliser

identité, client, route normalisée, décision d'autorisation, objet ciblé, statut, latence, quota, corrélation et résultat métier.

Ne pas journaliser

mots de passe, bearer tokens, clés API, données bancaires, contenu médical ou corps complets par défaut.

Détecter

séquences d'identifiants, hausse des 403/404, appels d'admin, exports massifs, variations de méthodes, erreurs de parsing et abus de flux métier.

- ▶ stocker dans un gestionnaire de secrets, jamais dans le dépôt ;
- ▶ attribuer une identité distincte à chaque service et environnement ;
- ▶ utiliser des durées courtes et une rotation automatisée ;
- ▶ limiter scopes, hôtes, opérations et quotas ;
- ▶ hacher les clés persistées lorsque la vérification le permet ;
- ▶ redacter logs, traces, erreurs et outils de support ;
- ▶ disposer d'une procédure de révocation et d'incident.

Références officielles principales

- ▶ OWASP API Security Top 10 – 2023 : <https://owasp.org/API-Security/editions/2023/en/0x11-t10/>
- ▶ OWASP API Security Project : <https://owasp.org/www-project-api-security/>
- ▶ Spring Security Resource Server :
<https://docs.spring.io/spring-security/reference/servlet/oauth2/resource-server/>
- ▶ Laravel Sanctum : <https://laravel.com/docs/sanctum>
- ▶ Laravel Rate Limiting : <https://laravel.com/docs/rate-limiting>
- ▶ FastAPI Security : <https://fastapi.tiangolo.com/tutorial/security/>
- ▶ FastAPI OAuth2/JWT : <https://fastapi.tiangolo.com/tutorial/security/oauth2-jwt/>

Usage

Vérifier la documentation correspondant à la version réellement déployée avant de copier une configuration.

SYNTHÈSE

Méthode, prévention et révision

Risque	Mesure principale	Vérification
Endpoint oublié	inventaire et retrait	comparer routes et documentation
Méthode cachée	autorisation par opération	tester chaque verbe avec faible privilège
Format alternatif	validation uniforme	rejouer JSON, formulaire, XML autorisés
Champ caché	allowlist/DTO	envoyer une propriété inconnue
Mass assignment	modèle d'entrée dédié	tester une propriété sensible
SSPP	constructeur typé	injecter séparateurs encodés

Cartographie

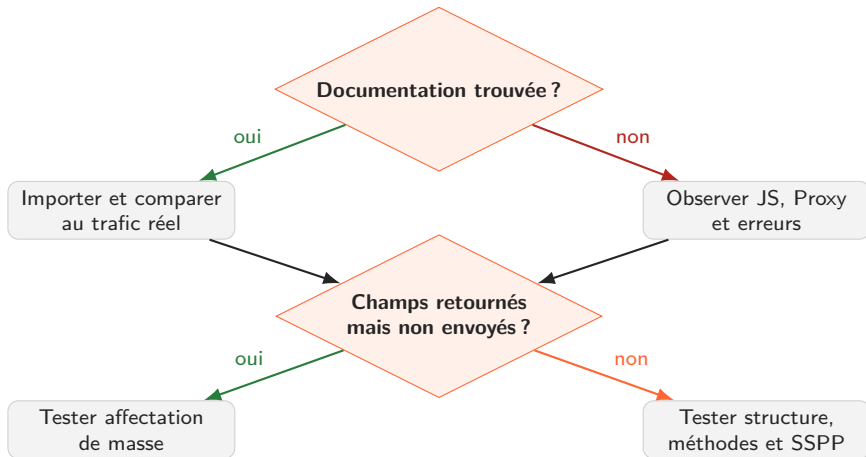
- ▶ hôtes, versions, documentation ;
- ▶ endpoints et méthodes ;
- ▶ paramètres et schémas ;
- ▶ rôles et objets ;
- ▶ dépendances internes.

Validation

- ▶ contrôle d'accès par objet/fonction ;
- ▶ champs cachés et types ;
- ▶ formats et doublons ;
- ▶ erreurs et limites ;
- ▶ persistance et effet métier.

Toujours

Conserver une preuve reproductible, minimale et non destructive.



- 1 Pourquoi une documentation exposée n'est-elle pas, seule, une vulnérabilité ?
- 2 Pourquoi OPTIONS ne suffit-il pas à lister toutes les méthodes réelles ?
- 3 Quelle différence existe entre paramètre caché et mass assignment ?
- 4 Pourquoi relire la ressource après une modification ?
- 5 Comment un %26 peut-il modifier une requête interne ?
- 6 Pourquoi les paramètres dupliqués sont-ils dangereux entre deux parseurs ?
- 7 Où faut-il appliquer l'autorisation dans une architecture à plusieurs services ?

- 1 Le défaut apparaît si une opération révélée manque de contrôle d'accès.
- 2 L'implémentation, le proxy ou la configuration peut rendre la réponse incomplète.
- 3 Un paramètre caché est non documenté ; la mass assignment lui donne un effet non autorisé par copie automatique.
- 4 Pour distinguer un simple écho d'une modification persistante.
- 5 Après décodage, il peut devenir un séparateur de paramètres dans la requête dorsale.
- 6 Chaque parseur peut choisir une valeur différente et créer une divergence de validation.
- 7 Sur chaque service et chaque opération sensible, indépendamment des contrôles du frontal.

Idée à retenir

Une API sûre ne dépend pas de ce que le client choisit d'afficher ou d'envoyer : elle valide, autorise et construit chaque opération côté serveur.

Attaques et défense des API

Comprendre les risques, tester avec
méthode, développer en sécurité

OWASP TOP 10

MULTI-FRAMEWORK

DEVSECOPS

Gana Faye | Sécurité des applications web